

Java Backend Interview Series

Sub-Chapter 1 a

Java 8 Features

Lambda . Streams . Optional . Date/Time . CompletableFuture

Motivation + Before/After Code + Import Blocks + 30 Q&A + Pitfalls + Tips

1 Lambda Expressions and Method References

1.1 Introduction

Before Java 8, passing behaviour required anonymous inner classes that were verbose and noisy. Lambda expressions let you pass a block of code as data, enabling functional-style programming. A lambda is an instance of any functional interface (an interface with exactly one abstract method).

1.2 The Problem — Anonymous Inner Class Verbosity

```
import java.util.List;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;

// BEFORE Java 8 -- sort a list: 5 lines, most are boilerplate
List<String> names = new ArrayList<>(Arrays.asList("Charlie", "Alice", "Bob"));
Collections.sort(names, new Comparator<String>() {
    @Override
    public int compare(String a, String b) {
        return a.compareTo(b);
    }
});
```

Before Java 8: anonymous Comparator

1.3 The Solution — Lambda Syntax

```
import java.util.Comparator;
import java.util.function.Supplier;

// AFTER Java 8 -- (params) -> expression OR (params) -> { block; }
Runnable r1 = () -> System.out.println("hello"); // no param
Comparator<String> c1 = (a, b) -> a.compareTo(b); // expression body
Comparator<String> c2 = (String a, String b) -> { // block body
    int cmp = a.compareTo(b);
    return cmp;
};

// Effectively final: captured locals must not be reassigned
String prefix = "Hello";

// prefix = "Hi"; // would make it NOT effectively final
Supplier<String> greeter = () -> prefix + " World";
```

```
// Sort with lambda vs method reference

names.sort((a, b) -> a.compareTo(b));

names.sort(Comparator.comparing(String::length)); // preferred
```

Lambda syntax and effectively-final rule

1.4 Four Kinds of Method References

Kind	Syntax	Lambda Equivalent	Example
Static method	Type::staticMethod	(args) -> Type.staticMethod(args)	Integer::parseInt
Unbound instance	Type::instanceMethod	(inst, args) -> inst.method(args)	String::toUpperCase
Bound instance	obj::instanceMethod	(args) -> obj.method(args)	System.out::println
Constructor	Type::new	(args) -> new Type(args)	ArrayList::new

```
import java.util.function.Function;
import java.util.function.Consumer;
import java.util.function.Supplier;
import java.util.List;
import java.util.ArrayList;

// 1. Static method reference

Function<String,Integer> parse = Integer::parseInt;
parse.apply("42"); // 42

// 2. Unbound instance -- first arg is the receiver

Function<String,String> upper = String::toUpperCase;
upper.apply("hello"); // "HELLO"

// 3. Bound instance -- receiver is fixed at capture time

Consumer<String> printer = System.out::println;
printer.accept("bound!");

// 4. Constructor reference

Supplier<List<String>> factory = ArrayList::new;

List<String> list = factory.get(); // new ArrayList<>()
```

All four method reference kinds

TIP: Prefer method references over lambdas when the body is a pure delegation. `x -> x.toUpperCase()` is clearer as `String::toUpperCase`. Use a lambda when there is additional logic.

WARNING: Lambdas can only capture effectively final local variables. To mutate a captured value use `AtomicInteger` or a single-element array -- but first reconsider whether the design is appropriate.

2 Functional Interfaces

2.1 Introduction

A functional interface has exactly one abstract method (SAM). `@FunctionalInterface` is optional but adds a compile-time check. Java 8 provides a rich set of built-in interfaces in `java.util.function` covering the most common patterns.

2.2 Core `java.util.function` Interfaces

Interface	Signature	Method	Use case
<code>Predicate<T></code>	<code>T -> boolean</code>	<code>test(T)</code>	Filtering, conditions
<code>Function<T,R></code>	<code>T -> R</code>	<code>apply(T)</code>	Transformation / mapping
<code>Consumer<T></code>	<code>T -> void</code>	<code>accept(T)</code>	Side effects, printing
<code>Supplier<T></code>	<code>() -> T</code>	<code>get()</code>	Lazy factory, default value
<code>BiFunction<T,U,R></code>	<code>(T,U) -> R</code>	<code>apply(T,U)</code>	Two inputs, one output
<code>UnaryOperator<T></code>	<code>T -> T</code>	<code>apply(T)</code>	Transform, same type
<code>BinaryOperator<T></code>	<code>(T,T) -> T</code>	<code>apply(T,T)</code>	Combine two of same type

2.3 Predicate -- Composing Conditions

```
import java.util.function.Predicate;
import java.util.List;

Predicate<String> notEmpty = s -> !s.isEmpty();
Predicate<String> shortWord = s -> s.length() <= 5;

// Compose with and / or / negate
Predicate<String> valid = notEmpty.and(shortWord);
Predicate<String> either = notEmpty.or(shortWord);
Predicate<String> notShort = shortWord.negate();

List<String> words = List.of("hi", "", "hello", "extraordinary", "ok");
words.stream().filter(valid).forEach(System.out::println);

// Output: hi hello ok

// Predicate.not (Java 11)
words.stream().filter(Predicate.not(String::isEmpty)).toList();
```

Predicate composition

2.4 Function -- andThen and compose

```
import java.util.function.Function;
import java.util.function.BiFunction;
```

```

Function<String,String> trim = String::trim;

Function<String,String> upper = String::toUpperCase;

Function<String,Integer> length = String::length;

// andThen: left then right (left-to-right composition)
Function<String,String> trimUpper = trim.andThen(upper);
trimUpper.apply(" hello "); // "HELLO"

// Full chain
Function<String,Integer> pipeline = trim.andThen(upper).andThen(length);
pipeline.apply(" hello "); // 5

// BiFunction -- two inputs
BiFunction<String,Integer,String> repeat = (s,n) -> s.repeat(n);
repeat.apply("ab", 3); // "ababab"

```

Function andThen chains and BiFunction

2.5 Consumer and Supplier

```

import java.util.function.Consumer;
import java.util.function.Supplier;
import java.util.ArrayList;
import java.util.Optional;

// Consumer -- side effect, no return value
Consumer<String> print = System.out::println;

Consumer<String> log = s -> logger.info("Val: {}", s);

Consumer<String> both = print.andThen(log); // chain side effects
both.accept("hello");

// Supplier -- lazy factory, no input
Supplier<List<String>> factory = ArrayList::new;
List<String> list = factory.get();

// Optional.orElseGet uses Supplier -- only called if Optional is empty
String result = Optional.<String>empty()
    .orElseGet(() -> computeDefault());

```

Consumer chaining and Supplier lazy evaluation

WARNING: `Consumer.andThen()` chains side effects left-to-right. If the first `Consumer` throws, the second is not called. Handle exceptions explicitly in `Consumer` chains used for resource cleanup.

3 Stream API Deep Dive

3.1 Introduction

A Stream processes data in a functional pipeline. It is NOT a data structure -- it is a view over a source. Three stages: source, zero or more lazy intermediate operations, and one eager terminal operation. Streams are single-use and cannot be restarted.

3.2 Pipeline Anatomy: Source -> Intermediate -> Terminal

```
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

List<String> words = List.of("apple", "banana", "cherry", "apricot", "blueberry", "avocado");

// Nothing runs until terminal op is called

List<String> result = words.stream()
    .filter(w -> w.startsWith("a")) // intermediate (lazy)
    .map(String::toUpperCase) // intermediate (lazy)
    .sorted() // intermediate (lazy)
    .limit(2) // intermediate (lazy)
    .collect(Collectors.toList()); // terminal (eager) -- triggers all

// [APPLE, APRICOT]

// Short-circuit terminal -- stops early
Optional<String> first = words.stream()
    .filter(w -> w.length() > 6)
    .findFirst(); // "banana" -- rest of stream not processed
```

Stream pipeline: lazy evaluation and short-circuit

3.3 Common Intermediate Operations

```
import java.util.List;
import java.util.stream.Collectors;

// flatMap -- flatten nested collections (1:N)
List<List<String>> nested = List.of(List.of("a", "b"), List.of("c", "d"));
nested.stream()
    .flatMap(List::stream) // Stream<String>: a,b,c,d
    .forEach(System.out::print);

// distinct + sorted + peek (debug without consuming)
List.of(3,1,4,1,5,9,2,6,5).stream()
```

```

.distinct()

.sorted()

.peek(n -> System.out.print(n+ " "))

.filter(n -> n > 3)

.toList();

// mapToInt -- primitive stream, no boxing overhead
int totalLength = words.stream()
    .mapToInt(String::length)
    .sum();

```

flatMap, distinct, mapToInt

3.4 Terminal Operations: reduce and collect

```

import java.util.List;
import java.util.Map;
import java.util.IntSummaryStatistics;
import java.util.stream.Collectors;

// reduce -- fold all elements into one
List<Integer> nums = List.of(1,2,3,4,5);
int sum = nums.stream().reduce(0, Integer::sum); // 15

// groupingBy (like SQL GROUP BY)
Map<Integer,List<String>> byLen = words.stream()
    .collect(Collectors.groupingBy(String::length));

// partitioningBy -- exactly two groups: true / false
Map<Boolean,List<String>> parts = words.stream()
    .collect(Collectors.partitioningBy(w -> w.length() > 6));

// joining -- combine strings
String csv = words.stream()
    .collect(Collectors.joining(", ", "[", "]"));
// "[apple, banana, cherry, ...]"

// IntSummaryStatistics
IntSummaryStatistics stats = nums.stream()
    .mapToInt(Integer::intValue).summaryStatistics();
// count=5, sum=15, min=1, max=5, average=3.0

```

reduce, groupingBy, partitioningBy, joining, statistics

3.5 Parallel Streams

```
import java.util.List;
import java.util.ArrayList;
import java.util.stream.Collectors;

// parallelStream -- ForkJoinPool.commonPool()

long count = words.parallelStream()
    .filter(w -> w.contains("a")).count();

// BAD: non-thread-safe accumulation
List<String> unsafe = new ArrayList<>();
words.parallelStream().filter(w->w.startsWith("a"))
    .forEach(unsafe::add); // race condition!

// CORRECT: use collect()
List<String> safe = words.parallelStream()
    .filter(w -> w.startsWith("a"))
    .collect(Collectors.toList());

// BAD: non-associative reduce with parallel stream
int wrong = nums.parallelStream().reduce(0, (a,b) -> a - b); // wrong!

// CORRECT: use associative operation
int correct = nums.parallelStream().reduce(0, Integer::sum);
```

Parallel streams: correct and incorrect usage

WARNING: Never use a shared mutable collection as the target of a parallelStream forEach. Use collect() which builds the result safely. Only use parallel streams for large, CPU-bound, stateless operations -- benchmark first.

4 Optional<T>

4.1 Introduction

Optional<T> is a container that may or may not hold a non-null value. It makes absence explicit in the return type, replacing "return null and check everywhere". Use Optional only as a method return type -- not for fields, parameters, or collections.

4.2 Creating Optional

```
import java.util.Optional;

Optional<String> empty = Optional.empty();

Optional<String> ofVal = Optional.of("hello"); // NPE if null

Optional<String> safe = Optional.ofNullable(null); // empty if null

// From a repository
Optional<User> userOpt = userRepository.findById(id);
```

Optional creation

4.3 Extracting Values Safely

```
import java.util.Optional;

Optional<String> opt = Optional.of("hello");

// orElse -- always evaluates argument (even if present)
String a = opt.orElse("default"); // "hello"

// orElseGet -- Supplier called only if empty (lazy)
String b = opt.orElseGet(() -> computeDefault()); // supplier not called

// orElseThrow -- NoSuchElementException or custom exception
String c = opt.orElseThrow();
String d = opt.orElseThrow(() -> new EntityNotFoundException("not found"));

// ifPresent, ifPresentOrElse (Java 9)
opt.ifPresent(val -> System.out.println("Found: " + val));
opt.ifPresentOrElse(
    val -> System.out.println("Found: " + val),
    () -> System.out.println("Not found"));
```

Optional extraction: orElse vs orElseGet vs orElseThrow

4.4 Transforming with map / flatMap / filter

```
import java.util.Optional;

// map -- transform value if present
```

```
Optional<String> nameOpt = userOpt.map(User::getName);

// flatMap -- when mapper itself returns Optional (avoid Optional<Optional<T>>)
Optional<Address> addrOpt = userOpt.flatMap(User::getAddress);

// filter -- empty if predicate fails
Optional<User> active = userOpt.filter(u -> u.isActive());

// Full null-safe chain -- no NPE anywhere
String city = findUser(id)
    .flatMap(User::getAddress)
    .map(Address::getCity)
    .filter(c -> !c.isBlank())
    .orElse("Unknown");
```

Optional map / flatMap / filter chain

WARNING: Never call `Optional.get()` without a prior `isPresent()` check -- it throws `NoSuchElementException` just like `NPE`. Use `orElseThrow()` with a meaningful message. Never store `Optional` in a field or pass it as a method parameter.

TIP: Prefer `orElseGet(supplier)` over `orElse(value)` when the default is expensive to compute. `orElse` always evaluates its argument even when the `Optional` holds a value.

5 Default and Static Interface Methods

5.1 Introduction

Before Java 8, adding a method to an interface was a breaking change for every implementing class. Default methods let interfaces provide a method body, enabling backward-compatible API evolution. Java 8 used this to add `stream()`, `forEach()`, and `sort()` to existing collection interfaces without breaking user code.

5.2 Default Methods -- Backward-Compatible Evolution

```
// Java 8 added stream() and forEach() to Collection via default methods

// without breaking ArrayList, LinkedList, etc.

interface Printable {

    String content(); // abstract -- must implement

    default void print() { // default -- optional to override
        System.out.println(content());
    }

    default void printWithBorder() {
        System.out.println("---");
        print();
        System.out.println("---");
    }
}

class Document implements Printable {

    private final String text;

    Document(String text) { this.text = text; }

    public String content() { return text; } // only abstract method needed

    // print() and printWithBorder() inherited for free
}
```

Default methods: interface with default implementations

5.3 Diamond Problem Resolution

```
interface A { default String hello() { return "Hello from A"; } }

interface B { default String hello() { return "Hello from B"; } }
```

```
// Implementing class MUST override when two interfaces conflict

class C implements A, B {

@Override

public String hello() {

return A.super.hello(); // explicitly call A's default

// or B.super.hello(); or a new implementation

}

}

// Resolution priority (highest first):

// 1. Class body always wins over any default method

// 2. More specific interface wins (sub-interface over super-interface)

// 3. Still ambiguous: must override explicitly (like C above)
```

Diamond problem resolution rule

5.4 Static Interface Methods

```
import java.util.Comparator;
import java.util.List;
import java.util.function.Predicate;

// Static methods belong to the interface, not instances

interface Validator<T> {

boolean validate(T value);

static <T> Validator<T> of(Predicate<T> p) { return v -> p.test(v); }

static <T> Validator<T> alwaysTrue() { return v -> true; }

}

Validator<String> emailVal = Validator.of(s -> s.contains("@"));

emailVal.validate("alice@example.com"); // true

// Comparator static methods (all added in Java 8)

Comparator<String> byLen = Comparator.comparingInt(String::length);

Comparator<String> sorted = byLen.thenComparing(Comparator.naturalOrder());

List<String> result = names.stream().sorted(sorted).toList();
```

Static interface methods and Comparator composition

TIP: Keep default methods short. Delegate to abstract methods so subclasses can customize by overriding only the abstract part. Default methods should feel like convenience wrappers, not core logic.

6 Date/Time API -- java.time

6.1 Introduction

java.util.Date and Calendar were mutable, not thread-safe, 0-indexed months, and hard to format correctly. Java 8 introduced java.time (inspired by Joda-Time): immutable, thread-safe, ISO-8601 by default, clearly named classes.

6.2 Core Classes

Class	Contains	Use case
LocalDate	Year, month, day	Birthdays, due dates (no time/zone)
LocalTime	Hour, minute, second, nano	Opening hours (no date/zone)
LocalDateTime	Date + time (no zone)	Timestamps in single timezone
ZonedDateTime	Date + time + ZoneId	Cross-timezone events, logging
Instant	Epoch-second + nano	Unique timestamps, DB storage
Period	Years, months, days	Age calculation, date arithmetic
Duration	Seconds + nanoseconds	Elapsed time, timeouts

6.3 LocalDate, LocalDateTime, ZonedDateTime

```
import java.time.LocalDate;

import java.time.LocalDateTime;

import java.time.ZonedDateTime;

import java.time.ZoneId;

import java.time.Period;

import java.time.Duration;

import java.time.Month;

LocalDate today = LocalDate.now();

LocalDate birthday = LocalDate.of(1990, Month.MARCH, 15);

LocalDate future = today.plusDays(30).plusMonths(2);

Period age = Period.between(birthday, today);

System.out.println(age.getYears() + " years old");

LocalDateTime dt = LocalDateTime.of(2024, 6, 15, 9, 30, 0);

LocalDateTime next = dt.plusHours(1);

// Convert to ZonedDateTime

ZoneId london = ZoneId.of("Europe/London");

ZonedDateTime londonTime = dt.atZone(london);

ZonedDateTime utcTime = londonTime.withZoneSameInstant(ZoneId.of("UTC"));
```

```
Duration elapsed = Duration.between(dt, next);

elapsed.toMinutes(); // 60
```

LocalDate, LocalDateTime, ZonedDateTime, Period, Duration

6.4 DateTimeFormatter -- Format and Parse

```
import java.time.LocalDate;
import java.time.Instant;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Locale;

LocalDate date = LocalDate.of(2024, 6, 15);

// Built-in formatters

String iso = date.format(DateTimeFormatter.ISO_LOCAL_DATE); // "2024-06-15"

// Custom pattern

DateTimeFormatter custom = DateTimeFormatter.ofPattern("dd/MM/yyyy");

String formatted = date.format(custom); // "15/06/2024"

LocalDate parsed = LocalDate.parse("15/06/2024", custom);

// Locale-aware

DateTimeFormatter locFmt = DateTimeFormatter
    .ofLocalizedDate(FormatStyle.LONG)
    .withLocale(Locale.UK);

date.format(locFmt); // "15 June 2024"

// Instant -- machine time

Instant now = Instant.now();

long epochSec = now.getEpochSecond();

Instant fromEpoch = Instant.ofEpochSecond(1717200000L);
```

DateTimeFormatter: format, parse, Instant

WARNING: `LocalDate`, `LocalTime`, `LocalDateTime` are immutable. Methods like `plusDays()` and `minusMonths()` return a NEW object -- they do NOT modify the original. Silently ignoring the return value is a common bug.

TIP: Store timestamps as `Instant` in databases (UTC epoch). Convert to `ZonedDateTime` only for display. Use `DateTimeFormatter.ISO_INSTANT` for REST API timestamp responses.

7 Collectors

7.1 Introduction

Collectors are reduction operations for `Stream.collect()`. They encapsulate the mutable fold pattern -- accumulate elements into a container, optionally transform. The Collectors factory class provides 40+ ready-made collectors.

7.2 Basic Collectors

```
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.stream.Collectors;

List<String> words = List.of("apple", "banana", "cherry", "avocado", "blueberry");

// toList / toSet / toUnmodifiableList

List<String> mutable = words.stream().collect(Collectors.toList());
Set<String> set = words.stream().collect(Collectors.toSet());
List<String> immut = words.stream().collect(Collectors.toUnmodifiableList());

List<String> short1 = words.stream().filter(w->w.length()>5).toList(); // Java 16+

// toMap -- key and value extractor
Map<String,Integer> wordLen = words.stream()
    .collect(Collectors.toMap(w -> w, String::length)); // throws on duplicate!

// toMap with merge function for duplicate keys
Map<Integer,String> merged = words.stream()
    .collect(Collectors.toMap(
        String::length,
        w -> w,
        (existing, next) -> existing + "," + next
    ));
```

toList, toSet, toMap with merge function

7.3 groupingBy and partitioningBy

```
import java.util.List;
import java.util.Map;
import java.util.IntSummaryStatistics;
import java.util.stream.Collectors;

// groupingBy -- group by key, collect to List<T> by default
Map<Integer,List<String>> byLen = words.stream()
```

```

.collect(Collectors.groupingBy(String::length));

// groupingBy with downstream -- count per group
Map<Integer, Long> countByLen = words.stream()
    .collect(Collectors.groupingBy(String::length, Collectors.counting()));

// groupingBy with joining downstream
Map<Integer, String> joinedByLen = words.stream()
    .collect(Collectors.groupingBy(
        String::length,
        Collectors.joining(", ")
    ));

// partitioningBy -- exactly {true: [...], false: [...]}
Map<Boolean, List<String>> parts = words.stream()
    .collect(Collectors.partitioningBy(w -> w.length() > 6));

IntSummaryStatistics stats = words.stream()
    .collect(Collectors.summarizingInt(String::length));

// count=5, sum=32, min=5, max=9, average=6.4

```

groupingBy, partitioningBy, summarizingInt

7.4 joining, counting, collectingAndThen

```

import java.util.List;
import java.util.Map;
import java.util.Collections;
import java.util.stream.Collectors;

// joining
String csv = words.stream()
    .collect(Collectors.joining(", ", "[", "]"));

// "[apple, banana, cherry, avocado, blueberry]"

// counting
long count = words.stream()
    .filter(w -> w.startsWith("a"))
    .collect(Collectors.counting()); // 2

// averagingInt, summingInt
double avgLen = words.stream().collect(Collectors.averagingInt(String::length));
int totalLen = words.stream().collect(Collectors.summingInt(String::length));

```

```
// collectingAndThen -- collect then post-process
List<String> unmod = words.stream()
    .collect(Collectors.collectingAndThen(
        Collectors.toList(),
        Collections::unmodifiableList
    ));

// teeing (Java 12) -- two collectors in one pass
Map.Entry<Long,Long> twoStats = words.stream()
    .collect(Collectors.teeing(
        Collectors.counting(),
        Collectors.filtering(w -> w.length() > 5, Collectors.counting()),
        Map::entry
    )); // Entry{key=5, value=2}
```

joining, counting, collectingAndThen, teeing

TIP: `groupingBy` with a downstream `Collector` is the Java equivalent of SQL `GROUP BY` + aggregate function. Think: `Collectors.groupingBy(classifier, Collectors.counting()) = SELECT key, COUNT(*) GROUP BY key.`

8 CompletableFuture

8.1 Introduction

`CompletableFuture<T>` extends `Future<T>` with a composable, non-blocking model. Unlike `Future.get()` which blocks, you attach callbacks (`thenApply`, `thenCompose`, etc.) that execute when the result is available, optionally on a custom thread pool.

8.2 Creating CompletableFuturees

```
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

// supplyAsync -- async computation, returns result
CompletableFuture<String> cf = CompletableFuture.supplyAsync(() -> {
    Thread.sleep(500);
    return fetchFromDatabase();
});

// runAsync -- async, no result
CompletableFuture<Void> fire = CompletableFuture.runAsync(
    () -> sendEmailNotification()
);

// Custom executor (recommended in production)
ExecutorService io = Executors.newFixedThreadPool(10);
CompletableFuture<String> withPool =
    CompletableFuture.supplyAsync(() -> fetchFromDatabase(), io);

// Already-completed (useful for tests)
CompletableFuture<String> done = CompletableFuture.completedFuture("cached");
```

Creating CompletableFuturees

8.3 Chaining: thenApply / thenCompose / thenCombine

```
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;

// thenApply -- transform result (like Stream.map)
CompletableFuture<Integer> length = cf.thenApply(String::length);

// thenApplyAsync -- transform on a different thread
CompletableFuture<Integer> lenAsync = cf.thenApplyAsync(String::length, io);
```

```
// thenCompose -- flat-map: mapper returns another CF
// avoids CompletableFuture<CompletableFuture<Order>>
CompletableFuture<Order> order = fetchUserId()
    .thenCompose(id -> fetchOrders(id))
    .thenCompose(o -> enrichWithItems(o));

// thenCombine -- zip two independent CFs when both complete
CompletableFuture<String> user = CompletableFuture.supplyAsync(() -> fetchUser());
CompletableFuture<String> account = CompletableFuture.supplyAsync(() -> fetchAccount());
CompletableFuture<String> combined = user.thenCombine(account,
    (u, a) -> "User: " + u + ", Account: " + a);

// thenAccept -- consume result, return CF<Void>
cf.thenAccept(r -> System.out.println("Result: " + r));
```

thenApply, thenCompose, thenCombine, thenAccept

8.4 Exception Handling: exceptionally / handle / whenComplete

```
import java.util.concurrent.CompletableFuture;

// exceptionally -- recover from failure, return fallback
CompletableFuture<String> safe = cf
    .exceptionally(ex -> {
        logger.error("Failed", ex);
        return "default-value";
    });

// handle -- runs on BOTH success and failure
CompletableFuture<String> handled = cf
    .handle((result, ex) -> {
        if (ex != null) return "Error: " + ex.getMessage();
        return result.toUpperCase();
    });

// whenComplete -- side effect, does NOT change the result
cf.whenComplete((result, ex) -> {
    if (ex != null) metrics.increment("cf.errors");
    else metrics.increment("cf.success");
});

// allOf -- wait for ALL CFs to complete
CompletableFuture<Void> all = CompletableFuture.allOf(user, account, cf);
```

```
all.thenRun(() -> System.out.println("All done!"));

// anyOf -- proceed when ANY completes (result is Object)
CompletableFuture<Object> any = CompletableFuture.anyOf(user, account);
any.thenAccept(first -> System.out.println("First: " + first));
```

exceptionally, handle, whenComplete, allOf, anyOf

WARNING: `CompletableFuture.get()` blocks the calling thread and wraps exceptions in `ExecutionException`. Prefer `thenApply/handle` chains. If you must block, use `join()` which throws unchecked `CompletionException`.

TIP: Always supply a custom `Executor` in production. The default `ForkJoinPool.commonPool()` is shared with parallel streams and `fork/join` tasks and can become a bottleneck under load.

9 Common Pitfalls and Best Practices

9.1 Stream Reuse

```
import java.util.stream.Stream;

// Streams are SINGLE-USE -- second terminal op throws

Stream<String> stream = words.stream().filter(w -> w.startsWith("a"));

long count1 = stream.count(); // OK

long count2 = stream.count(); // IllegalStateException: stream has already been operated upon

// Fix: recreate the stream

long c1 = words.stream().filter(w -> w.startsWith("a")).count();

long c2 = words.stream().filter(w -> w.startsWith("a")).count();
```

Stream reuse throws IllegalStateException

9.2 Parallel Stream Non-Associative Reduce

```
import java.util.List;

// BAD: subtraction is NOT associative -- wrong result with parallel

List<Integer> nums = List.of(1,2,3,4,5);

int wrong = nums.parallelStream().reduce(0, (a,b) -> a - b); // unpredictable!

// CORRECT: use associative operation

int correct = nums.parallelStream().reduce(0, Integer::sum); // always 15
```

Parallel stream requires associative operation

9.3 Optional.get() Anti-Pattern

```
import java.util.Optional;

// WRONG

Optional<String> opt = Optional.empty();

String val = opt.get(); // NoSuchElementException

// CORRECT

String val2 = opt.orElseThrow(() -> new UserNotFoundException("id=" + id));

// BEST -- chain without unwrapping

String city = userRepo.findById(id)

    .flatMap(User::getAddress)

    .map(Address::getCity)

    .orElse("Unknown");
```

Optional.get() is an anti-pattern

9.4 Date Mutability Assumption

```
import java.time.LocalDate;

// WRONG -- plus/minus return a NEW object, NOT mutate

LocalDate date = LocalDate.of(2024, 1, 1);

date.plusDays(30); // result discarded!

System.out.println(date); // still 2024-01-01

// CORRECT

LocalDate nextMonth = date.plusDays(30);

System.out.println(nextMonth); // 2024-01-31
```

LocalDate immutability -- capture the return value

WARNING: Do not confuse `Collectors.toList()` (mutable `ArrayList`) with `Stream.toList()` (unmodifiable, Java 16+). Calling `add/remove` on the result of `Stream.toList()` throws `UnsupportedOperationException` at runtime.

10 Interview Q and A -- Java 8 Features

30 questions covering standard and advanced interview depth. [ADV] marks senior-level questions.

Q: What is a lambda expression in Java 8?

A: A lambda is an anonymous function that implements a functional interface. Syntax: (params) -> expression or (params) -> { block; }. It lets you pass behaviour as data without verbose anonymous inner class syntax. The compiler generates an invokedynamic call instead of creating a separate .class file.

Q: What is a functional interface?

A: An interface with exactly one abstract method (SAM). @FunctionalInterface is optional but adds a compile-time check. Examples: Runnable (no params), Comparator<T> (compare), Predicate<T> (test), Function<T,R> (apply).

Q: What are the four types of method references?

A: 1) Static: Integer::parseInt maps to (s) -> Integer.parseInt(s). 2) Unbound instance: String::toUpperCase maps to (s) -> s.toUpperCase(). 3) Bound instance: System.out::println maps to (x) -> System.out.println(x). 4) Constructor: ArrayList::new maps to () -> new ArrayList<>().

Q: What does 'effectively final' mean in the context of lambdas?

A: A local variable never reassigned after first assignment. Lambdas can only capture effectively final locals because the value is copied into the lambda closure. The compiler checks this without requiring the final keyword explicitly.

Q: Difference between Predicate, Function, Consumer, Supplier?

A: Predicate<T>: T->boolean. Function<T,R>: T->R. Consumer<T>: T->void (side effect). Supplier<T>: ()->T (factory). They model the four fundamental shapes of computation in functional programming.

Q: What is the difference between map and flatMap in Stream?

A: map: 1:1 -- transforms each element to one result. flatMap: 1:N -- maps each element to a Stream, then flattens all streams into one. Use flatMap when your mapper returns a collection or Optional.

Q: Explain Stream lazy evaluation.

A: Intermediate operations (filter, map, sorted) build a pipeline description but execute nothing. The terminal operation (collect, count, findFirst) triggers evaluation. The JVM can fuse operations and short-circuit, avoiding materializing intermediate results.

Q: What does Optional.orElseGet() do differently from orElse()?

A: orElse(value) always evaluates its argument -- even when Optional is non-empty. orElseGet(supplier) only calls the Supplier if Optional is empty. Use orElseGet when the default is expensive to compute (e.g., a database call).

Q: When should you NOT use Optional?

A: Do not use Optional as a field type (extra object overhead, serialization issues), as a method parameter (callers must check for null before wrapping), or as a collection element (use empty collections instead of Optional-wrapped collections).

Q: What is a default method?

A: A method with a body declared in an interface using the default keyword. All implementing classes inherit it without being forced to override. Enables backward-compatible API evolution -- Java 8 added stream(), forEach(), sort() to existing collection interfaces this way.

Q: How is the diamond problem resolved with default methods?

A: The implementing class must override the ambiguous method. It can call A.super.methodName() or B.super.methodName() to delegate to a specific interface default. The compiler reports an error if both interfaces

define the same default and the class does not override.

Q: What is the difference between LocalDate and java.util.Date?

A: LocalDate: immutable, thread-safe, 1-indexed months (January=1), no time or timezone, clear API.
java.util.Date: mutable, not thread-safe, 0-indexed months, represents a millisecond timestamp (misleadingly named). Always use java.time classes for new code.

Q: What is the difference between Period and Duration?

A: Period is date-based (years, months, days). Duration is time-based (seconds, nanoseconds).
Period.between(date1, date2) gives age. Duration.between(time1, time2) gives elapsed time.

Q: Explain thenApply, thenCompose, thenCombine in CompletableFuture.

A: thenApply: transforms result (like Stream.map). thenCompose: flat-maps -- the mapper returns another CF, avoiding CF<CF<T>> nesting. thenCombine: waits for two independent CFs and merges results with a BiFunction.

Q: What does CompletableFuture.allOf() return?

A: CF<Void> that completes when ALL given CFs complete (or one fails). It does not carry individual results -- you must retrieve them from each original CF after allOf completes.

Q: How do you handle exceptions in a CompletableFuture chain?

A: exceptionally(fn): recovers from failure, returns fallback value. handle((result, ex) -> ...): runs on both success and failure, can transform either. whenComplete((result, ex) -> ...): side effect only, does not change the result.

Q: Difference between Collectors.toList() and Stream.toList()?

A: Collectors.toList() returns a mutable ArrayList. Stream.toList() (Java 16+) returns an unmodifiable list. add/remove on Stream.toList() throws UnsupportedOperationException. Stream.toList() also allows null elements (unlike List.copyOf).

Q: What is Collectors.groupingBy with a downstream collector?

A: Groups elements by a key (like SQL GROUP BY), then applies a second collector to each group. Example: groupingBy(String::length, Collectors.counting()) produces Map<Integer, Long> of length to count per group.

Q: What are short-circuit operations in Stream?

A: Operations that stop processing once the result is determined: findFirst(), findAny(), anyMatch(), allMatch(), noneMatch(), limit(n). They enable efficient lazy evaluation over potentially infinite streams.

Q: What is Comparator.thenComparing() and how does it work?

A: Returns a composed Comparator that first compares by the primary key, then uses the secondary Comparator only when the primary comparison returns 0 (a tie). This is a default method in Comparator that chains Comparators without writing nested if-else.

Q: [ADV] Explain how invokedynamic enables lambda expressions.

A: The lambda expression compiles to an invokedynamic call. On first invocation, LambdaMetafactory bootstrap generates a class implementing the functional interface using MethodHandles. Subsequent calls reuse the generated class. No extra .class file is created at compile time -- the class is generated lazily at runtime.

Q: [ADV] What is a Spliterator and how do parallel streams use it?

A: Spliterator<T> provides tryAdvance() (consume one element) and trySplit() (split for parallel work). ForkJoinPool calls trySplit() recursively to partition the data. Custom Spliterators enable parallel streams over non-standard sources like database result sets or custom data structures.

Q: [ADV] Explain the identity value requirement in Stream.reduce().

A: reduce(identity, accumulator): the identity must satisfy identity op x = x for all x (e.g., 0 for sum, 1 for product). In parallel streams, each fork starts with the identity -- a wrong identity (e.g., 1 for sum) adds one extra per fork, giving wrong results.

Q: [ADV] Difference between `thenApply` and `thenApplyAsync`?

A: `thenApply` runs the callback on the thread that completed the CF (or calling thread if already done). `thenApplyAsync` submits to `ForkJoinPool.commonPool()` or a supplied `Executor`. Use Async variants for I/O-bound or slow callbacks to avoid blocking the completing thread.

Q: [ADV] What is `Collectors.teeing()` (Java 12)?

A: Applies two downstream collectors simultaneously to the same stream elements, then merges their results. This avoids two passes over the data. Example: `Collectors.teeing(Collectors.counting(), Collectors.summingInt(String::length), Map::entry)` computes count and total length in one pass.

Q: [ADV] What happens when `Optional.map()` mapper returns null?

A: `Optional.map()` wraps the mapper result with `Optional.ofNullable()`, so returning null produces `Optional.empty()` -- no `NullPointerException`. `Optional.flatMap()` is different: the mapper must return `Optional` explicitly, and returning null throws `NullPointerException`.

Q: [ADV] Why are `IntStream/LongStream/DoubleStream` preferred for numeric work?

A: `Stream<Integer>` boxes each int in an `Integer` object causing heap allocation and GC pressure. Primitive streams (`IntStream`) operate on primitive values directly without boxing. For large numeric pipelines, `mapToInt().sum()` is significantly faster and uses less memory than `map().reduce(0, Integer::sum)`.

Q: [ADV] Why is `DateTimeFormatter` thread-safe but `SimpleDateFormat` is not?

A: `DateTimeFormatter` is immutable: no mutable state between calls. `SimpleDateFormat` stores intermediate parse state in instance fields, making concurrent access produce corrupted results or wrong dates. `SimpleDateFormat` must be wrapped in `ThreadLocal` or recreated per call.

Q: [ADV] Explain how `Comparator.comparing()` uses type inference.

A: `Comparator.comparing(keyExtractor)` takes a `Function<T,U extends Comparable<U>>` and returns `Comparator<T>`. The compiler infers `T` from usage context and `U` from the key extractor's return type. `thenComparing()` is a default method that returns a new `Comparator` delegating to the chained one when the primary comparison is 0.

Q: [ADV] Difference between `CompletableFuture.handle()` and `exceptionally()`?

A: `exceptionally(fn)` only runs on failure and provides a recovery value. `handle((result, ex) -> ...)` always runs on both success and failure -- it receives either the result or the exception and returns the new value. `handle()` is more general and can transform success results too, unlike `exceptionally()`.

11 Summary -- Java 8 Features Cheat Sheet

Feature	Key Points	Watch Out For
Lambda	SAM instance; captures effectively-final vars; invokedynamic	this = enclosing class; cannot capture mutable locals
Method Ref	4 types: static, unbound, bound, constructor	Unbound takes instance as first arg (easy to confuse with bound)
Predicate/Function	Compose with and/or/negate, andThen/compose	andThen left-to-right; compose right-to-left
Stream	Lazy pipeline; single-use; prefer primitive streams for numerics	Cannot reuse; parallel needs associative stateless ops
Optional	Return type only; use map/flatMap/orElseGet chains	Never use get() blindly; never use as field or parameter
Default Methods	Backward-compatible interface evolution	Ambiguity with two interfaces: must override with X.super.m()
java.time	Immutable; LocalDate/LocalDateTime/ZonedDateTime/Instant	All modifiers return new object -- capture the return value!
Collectors	groupingBy, partitioningBy, toMap, joining, teeing	toMap throws on duplicate keys without merge function
CompletableFuture	Non-blocking chains; handle exceptions with handle()	Use custom Executor; get() blocks; allOf returns CF<Void>